Linear Array (One dimension array)

A linear array, is a list of finite numbers of elements stored in the memory. In a linear array, we can store only homogeneous data elements. Elements of the array form a sequence or linear list, that can have the same type of data.

Each element of the array, is referred by an index set. And, the total number of elements in the array list, is the length of an array. We can access these elements with the help of the index set.

Example Array

- **For example, consider an array of employee names with the index set from 0 to 7, which contains 8 elements as shown in fig:**
-

Mike	Rick	christ	Alex	Max	Dave	Roly	Daina
0	1	2	3	4	5	6	7

- **Now, if we want to retrieve the name 'Alex' from the given array, we can do so by calling "employees [3]". It gives the element stored in that location, that is 'Alex', and so on.**

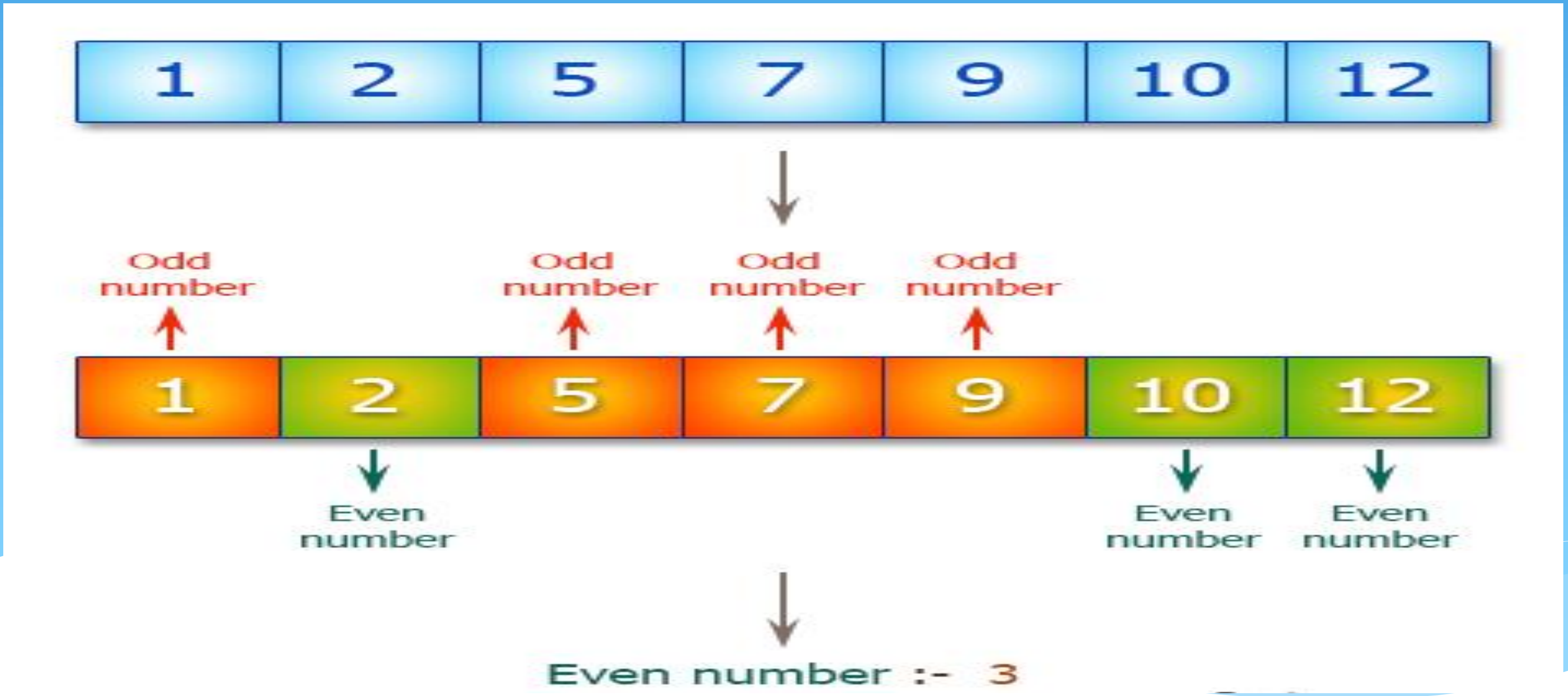
What is Algorithm

An algorithm is well defined list of steps to solve a particular problem. The time and space it uses are two major measures of the efficiency of an algorithm

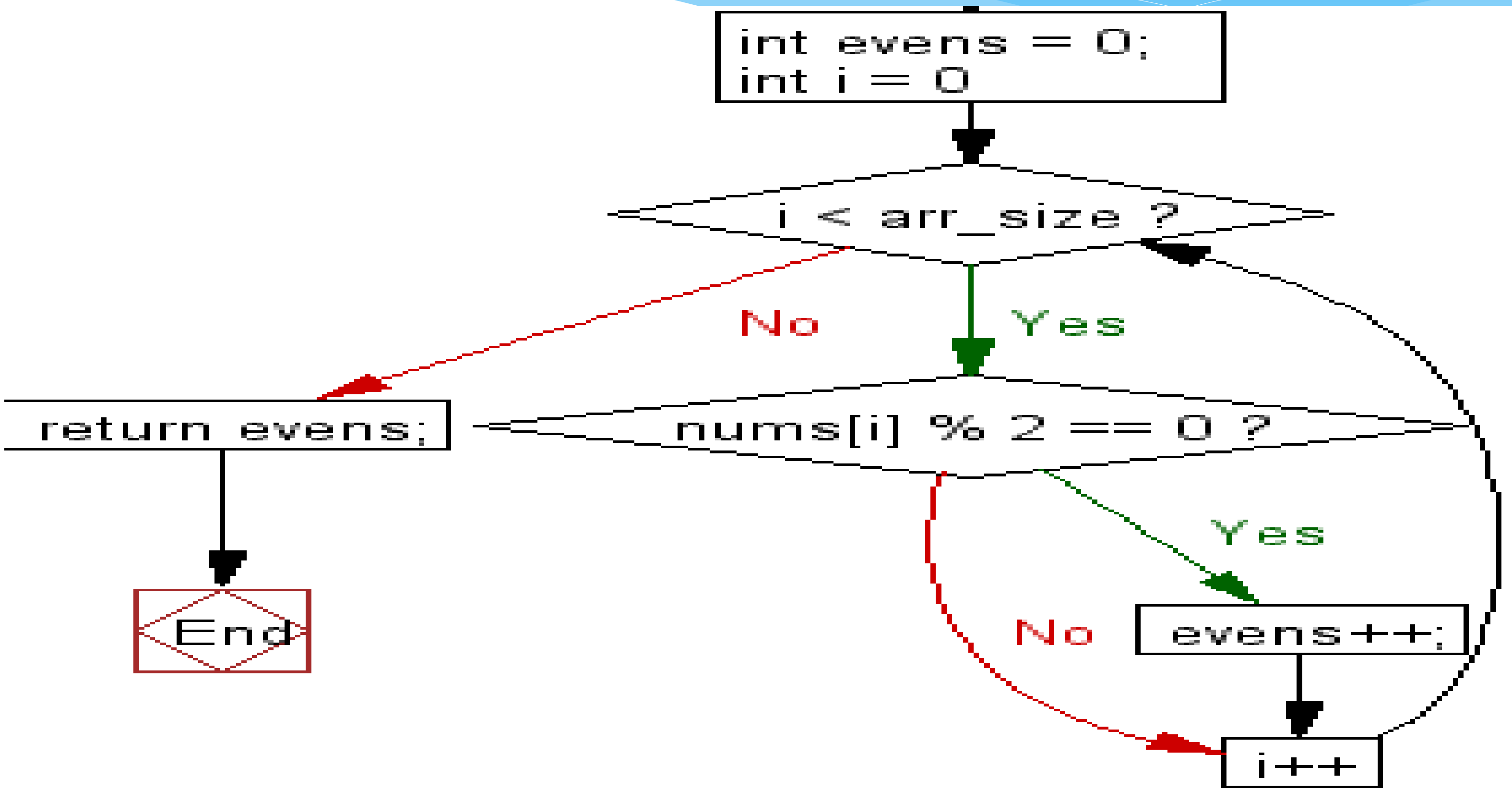
Traversing

Accessing each record Exactly once.

#Count Number of evens in an array



Traversing


int evens = 0;
int i = 0
i < arr_size ?
No
Yes
return evens;
nums[i] % 2 == 0 ?
End
Yes
No
evens++;
i++

Traversing

Algorithm:

1. [Initialization] Set evens=0
2. Repeat for i= 0 to less than array_size

    If nums [i] % 2==0 , Then

    Set evens=evens+1

    [End of loop]
3. Return.

Traversing

Accessing each record Exactly once.

Consider an array **AUTO** which records the number of automobiles sold each year from 1982 to 2020. Find the number **NUM** of years during which more than 300 automobiles were sold.

Algorithm:

1. [Initialization] Set NUM=0
2. Repeat for k= 1982 to 2020

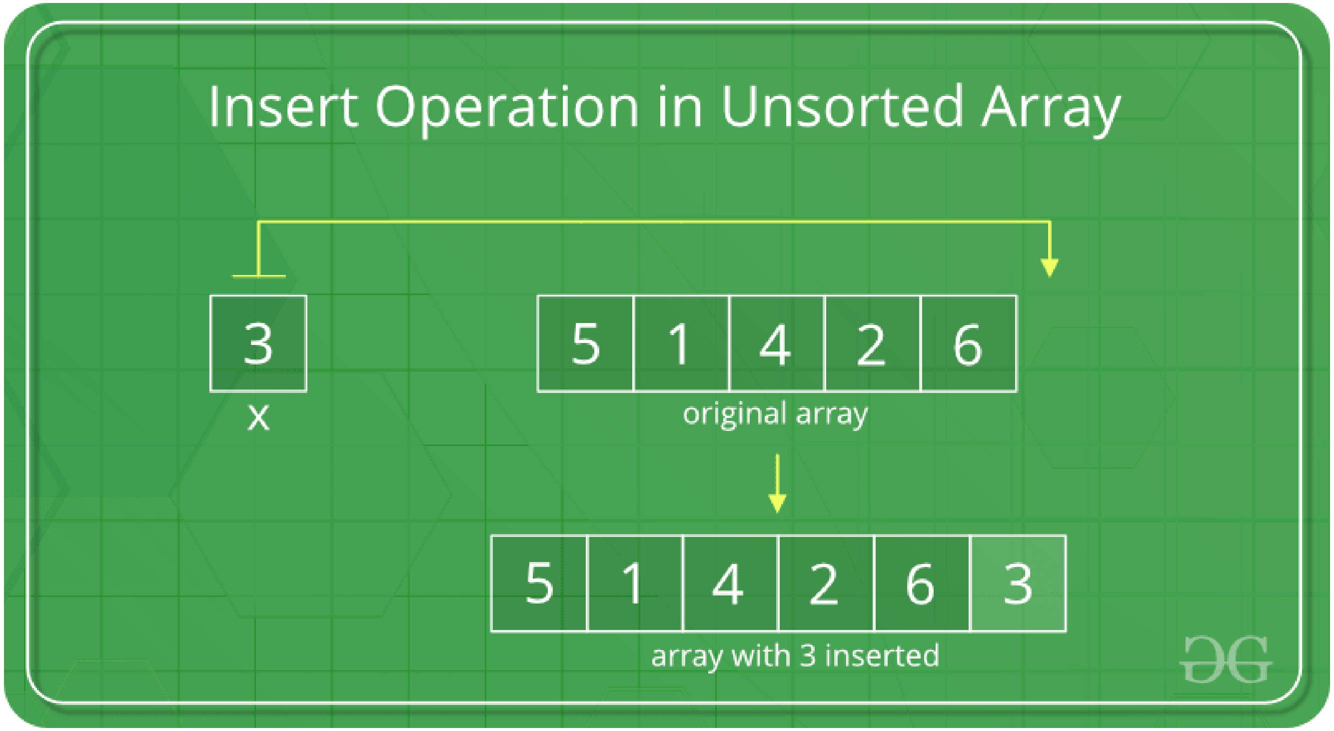
   If AUTO[k] > 300, Then

   Set NUM=NUM+1

   [End of loop]
3. Return.

Inserting

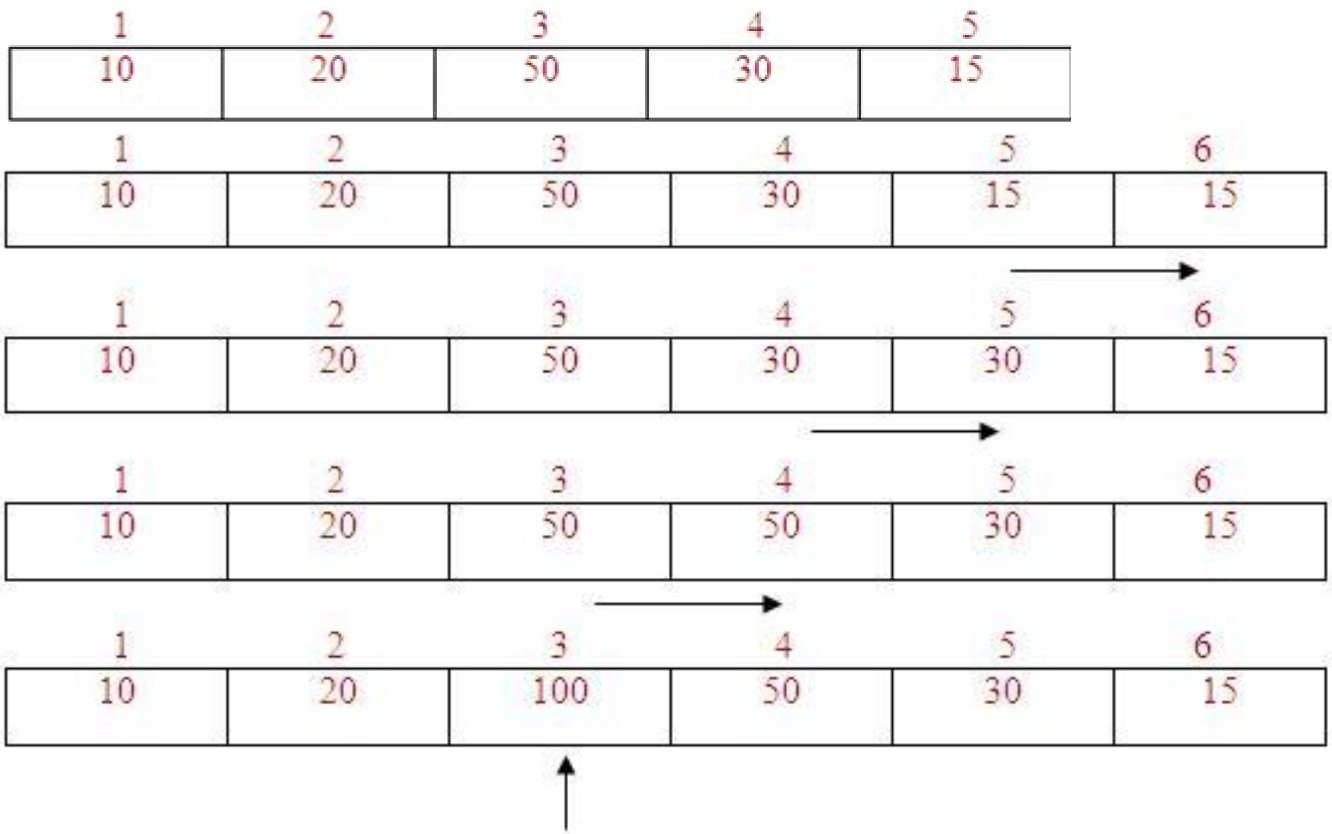
Add New record



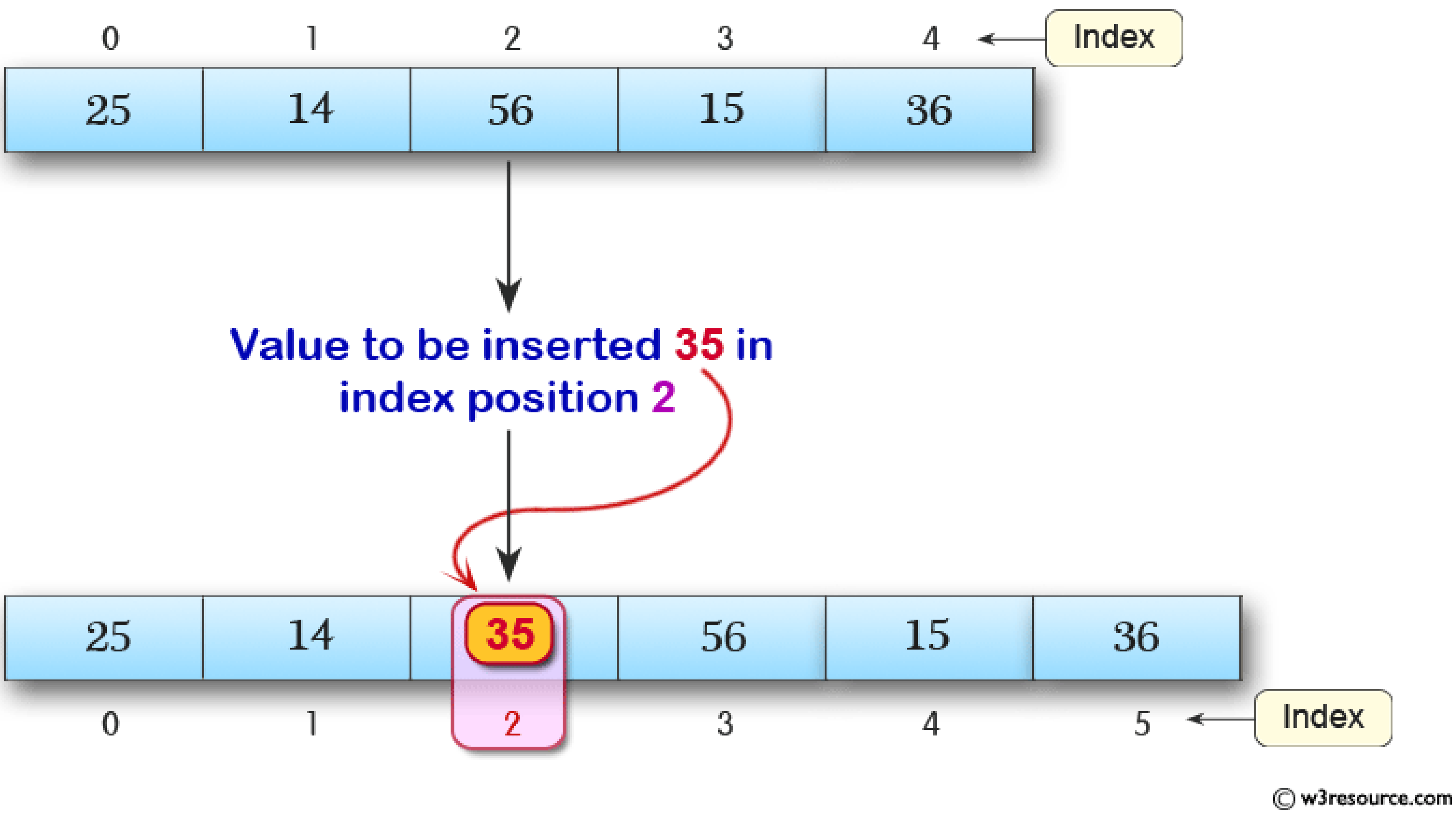
Inserting

Insert 100 into location 3

Shift all the elements from the last index to position index by 1 position to the right. insert the new element in array[position]



Inserting



0 1 2 3 4
Index
25 14 56 15 36
Value to be inserted 35 in
index position 2
25 14 35 56 15 36
0 1 2 3 4 5
Index
© w3resource.com

Inserting Algorithm

Algorithm 4.2: (Inserting into a Linear Array) INSERT (LA, N, K, ITEM)

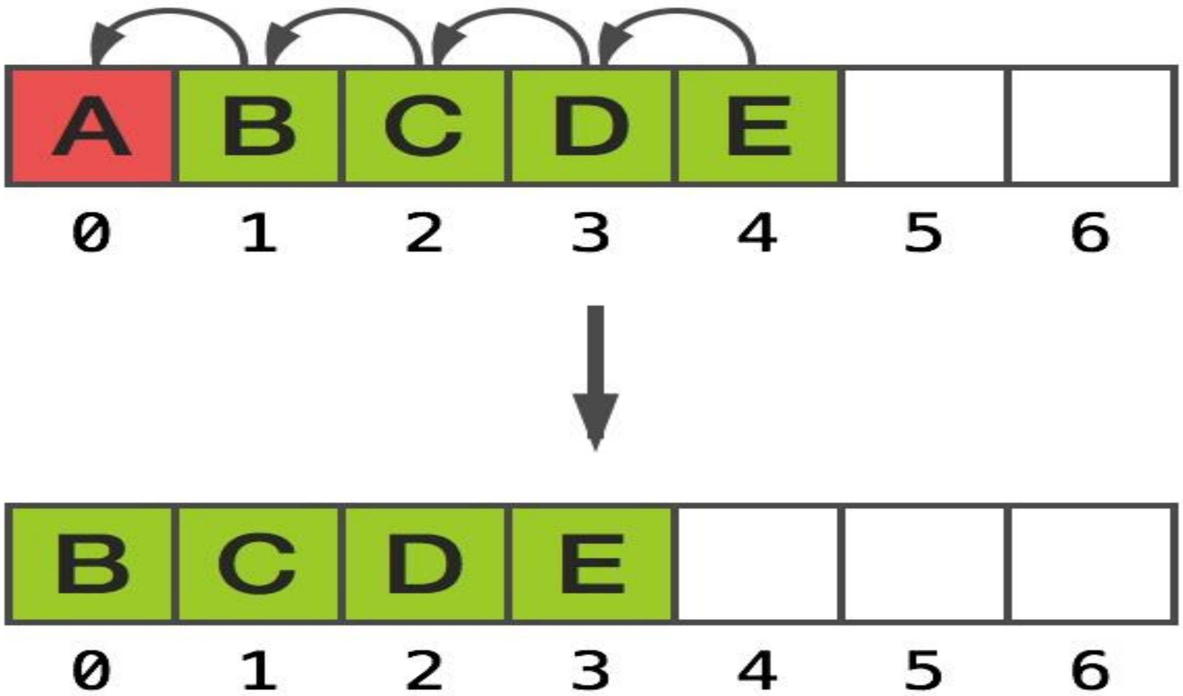
Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm inserts an element ITEM into the Kth position in LA.

1. [Initialize counter.] Set J : = N.
2. Repeat Steps 3 and 4 while $J \geq K$.
3. [Move Jth element downward.] Set LA[J + 1] := LA[J].
4. [Decrease counter.] Set J := J – 1.

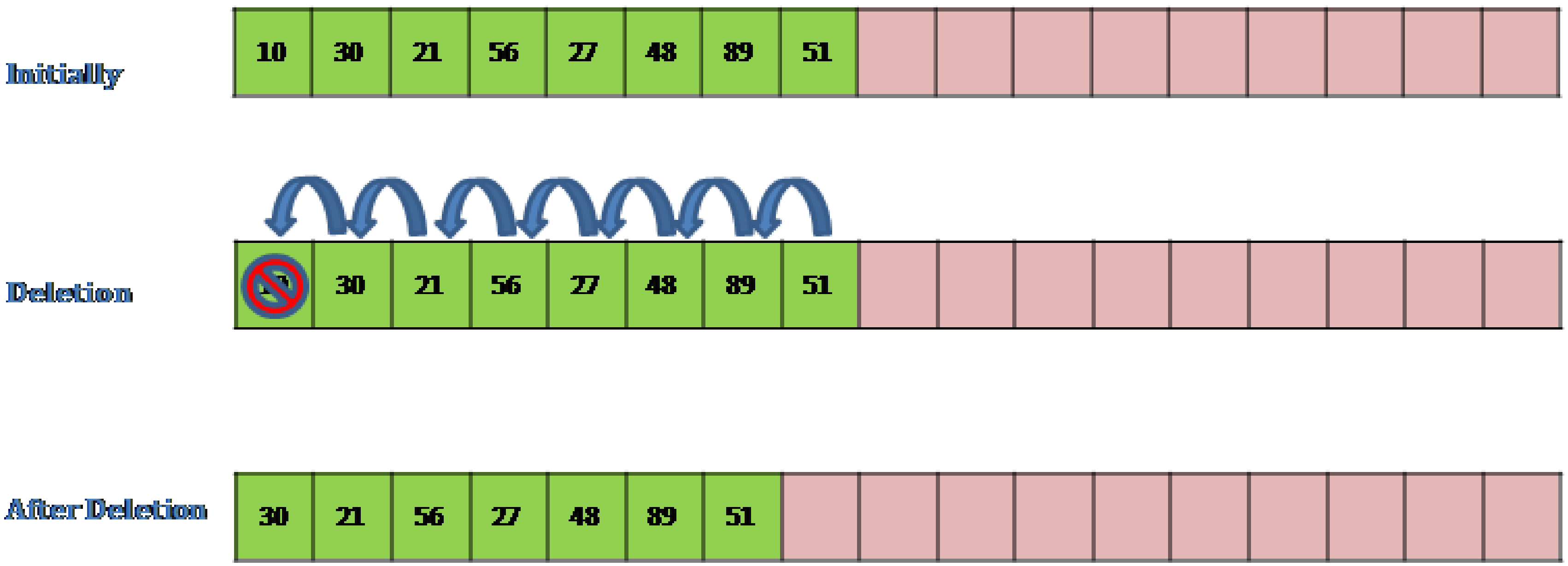
   [End of Step 2 loop.]
5. [Insert element.] Set LA[K] := ITEM.
6. [Reset N.] Set N := N + 1.
7. Exit.

Deleting

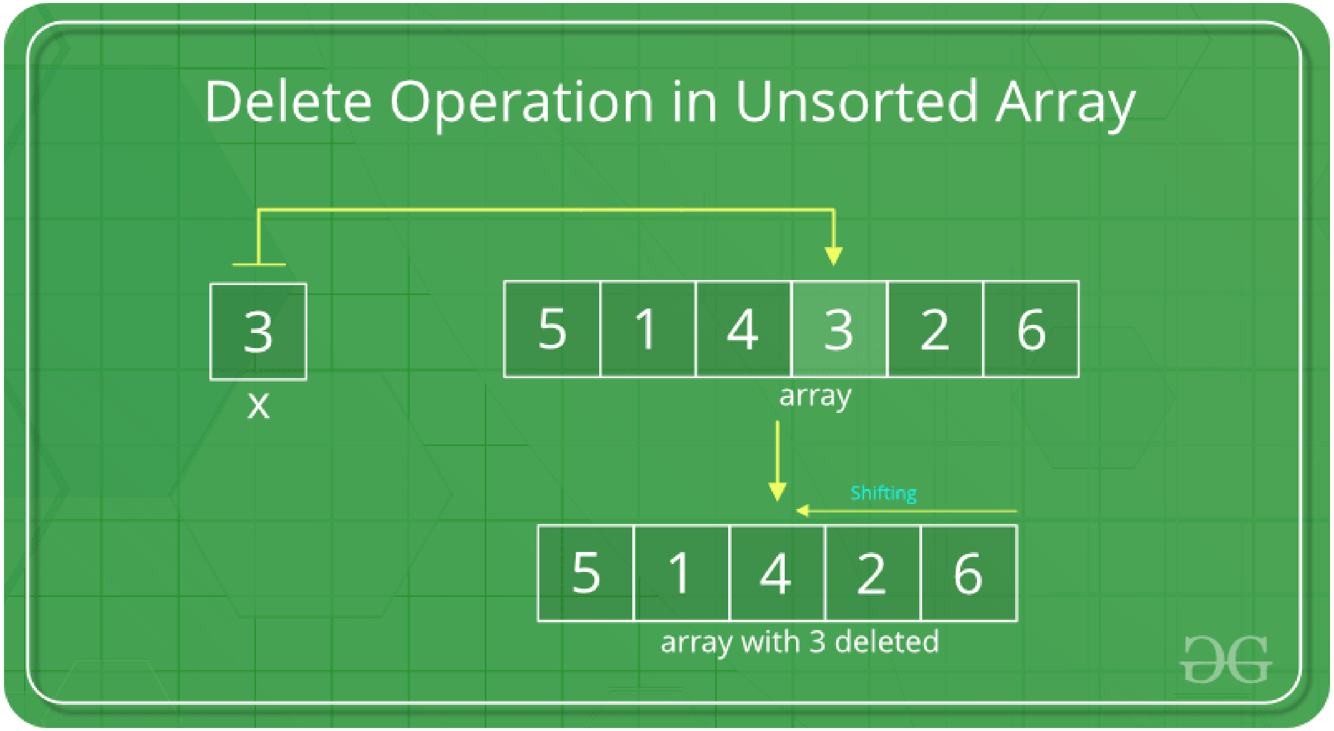
Remove record from list



Deleting ..


Initially
10 30 21 56 27 48 89 51
Deletion
30 21 56 27 48 89 51
After Deletion
30 21 56 27 48 89 51

Deleting..



Delete Operation in Unsorted Array
3
x
5 1 4 3 2 6
array
Shifting
5 1 4 2 6
array with 3 deleted

Deleting Algorithm

- **DELETE (LA, N , K , ITEM)** [LA is a linear array with N elements and K is a positive integers such that K ≤ N. This algorithm deletes K^{th} element from LA]
  1. Set ITEM := LA[K]
  2. Repeat for J = K to N -1:
     [Move the J + 1^{st} element upward] Set LA[J] := LA[J + 1]
  3. [Reset the number N of elements] Set N := N - 1;
  4. Exit